Algoritmos de Ordenación Elementales

Algoritmos y Estructuras de Datos

Rodrigo Pérez Rodríguez

Universidad Rey Juan Carlos

Contenido

1. Introducción
2. Interfaz Comparable en Java
3. Selection Sort
4. Insertion Sort
5. Shellsort
6. Conclusiones

Introducción

El problema de la ordenación

Definición:

- Reorganizar un array de N elementos en orden ascendente
- Cada elemento tiene una **clave** que determina el orden
- Los elementos pueden tener información adicional asociada

Ejemplo: Registros de estudiantes

Nombre	Curso	Nota	Email
Chen	3	A	chen@urjc.es
Furia	1	A	furia@urjc.es
Rohde	2	A	rohde@urjc.es

Clave: curso; queremos ordenar por curso

Ejemplo: ordenar números aleatorios

Ejemplo en Java:

```
public class Experiment {
    public static void main(String[] args) {
        int N = Integer.parseInt(args[0]);
        Double[] a = new Double[N];
        for (int i = 0; i < N; i++)
            a[i] = StdRandom.uniform();
        Insertion.sort(a);
        for (int i = 0; i < N; i++)
            System.out.println(a[i]);
    }

}
```

Ejemplo: ordenar cadenas de texto

Queremos ordenar una lista de palabras alfabéticamente:

```
public class StringSorter {

    public static void main(String[] args) {
        String[] a = StdIn.readAllStrings();
        Insertion.sort(a);
        for (String s : a)
                System.out.println(s);
    }
}
```

Salida: [kiwi, manzana, pera, uva]

Ejemplo: ordenar ficheros

Podemos leer datos de un fichero y ordenarlos:

```
public class FileSorter{

    public static void main(String[] args){
                File directory = new File(args[0]);
                File[] files = directory.listFiles();
                assert files != null;
                Insertion.sort(files);
                for (File file : files) System.out.println(file.getName());
    }
}
```

Permite ordenar ficheros fácilmente.

Aplicaciones de la ordenación

La ordenación es fundamental en computación:

- **Bases de datos:** Ordenar registros por diferentes criterios
- **Búsqueda:** Un array ordenado permite búsqueda eficiente
- **Interfaces de usuario:** Mostrar datos ordenados
- **Procesamiento de datos:** Detectar duplicados, encontrar medianas
- **Algoritmos:** Muchos algoritmos requieren datos ordenados

Ejemplos cotidianos:

- Contactos en el teléfono (alfabético)
- Mensajes de correo (por fecha)
- Productos en tienda online (por precio)
- Archivos en un directorio (por nombre)

Orden total

Para que la ordenación tenga sentido, necesitamos un orden total:

Una relación binaria $\leq$ que satisface:

1. **Antisimetría:** Si $v \leq w$ y $w \leq v$, entonces $v = w$
2. **Transitividad:** Si $v \leq w$ y $w \leq x$, entonces $v \leq x$
3. **Totalidad:** Para cualesquiera v, w: o bien $v \leq w$ o bien $w \leq v$

Ejemplos de orden total:

- Números naturales y reales (orden estándar)
- Fechas y tiempos (orden cronológico)
- Cadenas de texto (orden alfabético)

Contraejemplos (NO son orden total):

- Piedra-papel-tijera (viola transitividad)
- Relación de prerequisitos en asignaturas (viola totalidad)

Interfaz Comparable en Java

El problema: ordenar cualquier tipo de dato

Desafío:

- Queremos escribir un método `sort()` que funcione con cualquier tipo
- ¿Cómo puede `sort()` comparar objetos de tipo `String`, `Integer`, `Date`, etc.?
- No conocemos a priori qué tipos serán ordenados

Solución: Callbacks

- El cliente pasa un array de objetos a `sort()`
- `sort()` llama al método `compareTo()` de cada objeto
- Los objetos 'saben' cómo compararse entre sí

Implementación en Java:

- Usamos **interfaces**
- Los tipos implementan la interfaz `Comparable`

La interfaz Comparable

API de Comparable:

```
public interface Comparable<Item>
{
    public int compareTo(Item that);
}
```

Contrato de `compareTo()`:

Para `v.compareTo(w)`:

- Devuelve un entero **negativo** si $v < w$
- Devuelve **cero** si $v = w$
- Devuelve un entero **positivo** si $v > w$
- Lanza una excepción si los tipos son incompatibles
- Define un orden total

Tipos que ya implementan Comparable: `Integer`, `Double`, `String`, `Date`, `File`, ...

Ejemplo: Implementar Comparable

Tipo de dato Date:

```
public class Date implements Comparable<Date>
{
    private final int day, month, year;

    public Date(int d, int m, int y) {
        day = d; month = m; year = y;
    }

    public int compareTo(Date that) {
        if (this.year  < that.year)  return -1;
        if (this.year  > that.year)  return +1;
        if (this.month < that.month) return -1;
        if (this.month > that.month) return +1;
        if (this.day   < that.day)   return -1;
        if (this.day   > that.day)   return +1;
        return 0;
    }
}
```

Nota: Solo comparamos fechas con otras fechas (tipo genérico)

Cliente: usar un algoritmo de ordenación

Ejemplo de cliente:

```
public class StringSorter
{
    public static void main(String[] args)
    {
        String[] a = StdIn.readAllStrings();
        Insertion.sort(a);
        for (int i = 0; i < a.length; i++)
            System.out.println(a[i]);
    }
}
```

Punto clave:

- El código de ordenación NO depende del tipo específico
- Solo requiere que el tipo implemente `Comparable`
- Funciona con `String`, `Integer`, `Date`, tipos propios, etc.

Selection Sort

Selection Sort: idea

Algoritmo:

1. Encontrar el elemento más pequeño del array
2. Intercambiarlo con el primer elemento
3. Encontrar el segundo elemento más pequeño
4. Intercambiarlo con el segundo elemento
5. Continuar hasta que todo el array esté ordenado

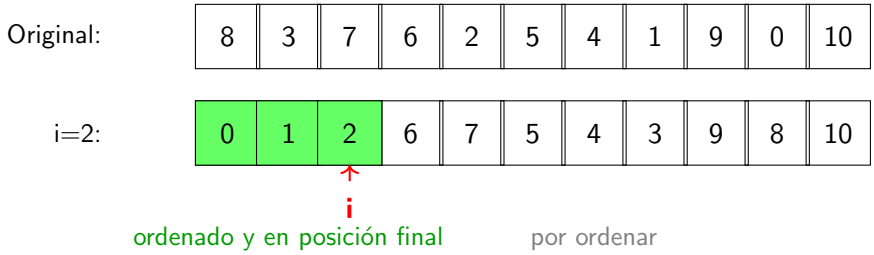
En la iteración i:

- Encontrar el índice `min` del elemento más pequeño a la derecha de i
- Intercambiar `a[i]` con `a[min]`

Selection Sort: invariantes

Invariantes del algoritmo:

- Las entradas a la **izquierda** del puntero (incluyéndolo) están:
  - En orden ascendente
  - En su posición final
  - Nunca se moverán de nuevo
- Ninguna entrada a la **derecha** del puntero es menor que ninguna a la izquierda (incluida)



Funciones auxiliares

Para escribir código de ordenación limpio:

Comparar: ¿Es v menor que w?

```
private static boolean less(Comparable v, Comparable w)
{
    return v.compareTo(w) < 0;
}
```

Intercambiar: Intercambiar elementos en posiciones i y j

```
private static void exch(Comparable[] a, int i, int j)
{
    Comparable temp = a[i];
    a[i] = a[j];
    a[j] = temp;
}
```

Toda la manipulación de datos se hace a través de `less()` y `exch()`

Selection Sort: implementación

```
public class Selection
{
    public static void sort(Comparable[] a)
    {
        int N = a.length;
        for (int i = 0; i < N; i++)
        {
            int min = i;
            for (int j = i+1; j < N; j++)
                if (less(a[j], a[min]))
                    min = j;
            exch(a, i, min);
        }
    }

    private static boolean less(Comparable v, Comparable w)
    { return v.compareTo(w) < 0; }

    private static void exch(Comparable[] a, int i, int j)
    { Comparable temp = a[i]; a[i] = a[j]; a[j] = temp; }
}
```

Selection Sort: bucle interno

Para mantener los invariantes:

Paso 1: Mover el puntero a la derecha

- `i++;`

Paso 2: Identificar el índice del mínimo a la derecha

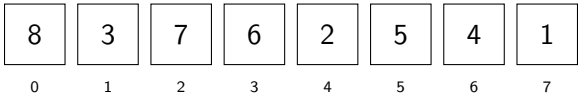
```
int min = i;
for (int j = i+1; j < N; j++)
    if (less(a[j], a[min]))
        min = j;
```

Paso 3: Intercambiar en posición

- `exch(a, i, min);`

Selection Sort: Animación paso a paso

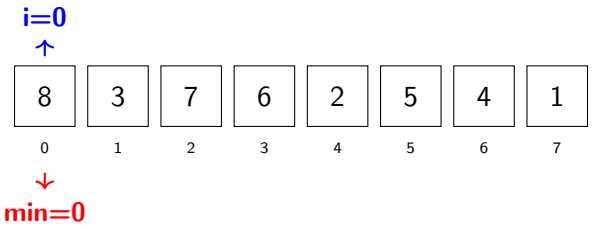
Estado inicial



Observa: i marca posición a rellenar, j busca mínimo, min guarda índice del menor

Selection Sort: Animación paso a paso

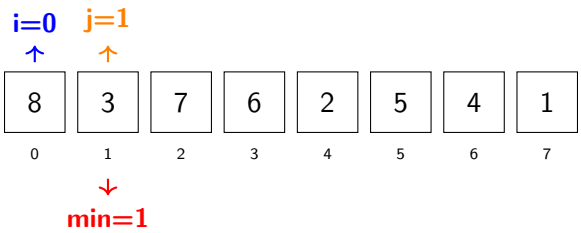
`min = i = 0;` buscar mínimo desde i+1



Observa: i marca posición a rellenar, j busca mínimo, min guarda índice del menor

Selection Sort: Animación paso a paso

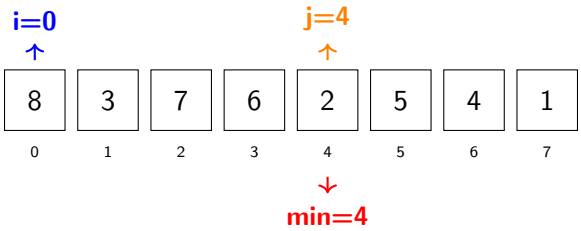
Comparar: a[j=1]=3 $<$ a[min=0]=8? Sí $\rightarrow$ min=1



Observa: i marca posición a rellenar, j busca mínimo, min guarda índice del menor

Selection Sort: Animación paso a paso

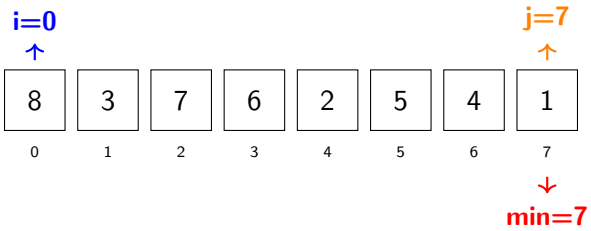
Comparar: a[j=4]=2 $<$ a[min=1]=3? Sí $\rightarrow$ min=4



Observa: i marca posición a rellenar, j busca mínimo, min guarda índice del menor

Selection Sort: Animación paso a paso

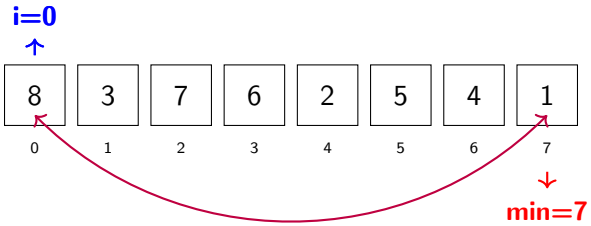
Comparar: a[j=7]=1 $<$ a[min=4]=2? Sí $\rightarrow$ min=7



Observa: i marca posición a rellenar, j busca mínimo, min guarda índice del menor

Selection Sort: Animación paso a paso

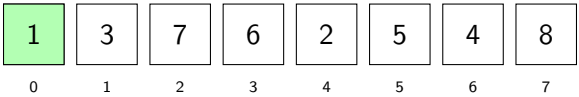
`exch(a, i=0, min=7)` → intercambia 8 y 1



Observa: i marca posición a rellenar, j busca mínimo, min guarda índice del menor

Selection Sort: Animación paso a paso

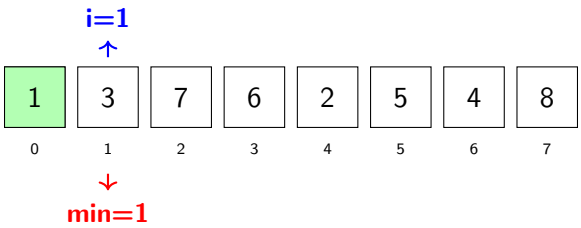
Posición 0 ordenada (a[0]=1). **i++**



Observa: i marca posición a rellenar, j busca mínimo, min guarda índice del menor

Selection Sort: Animación paso a paso

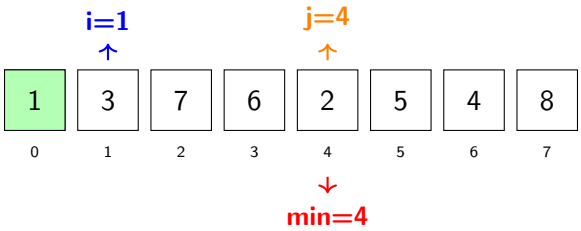
`min = i = 1;` buscar desde j=2



Observa: i marca posición a rellenar, j busca mínimo, min guarda índice del menor

Selection Sort: Animación paso a paso

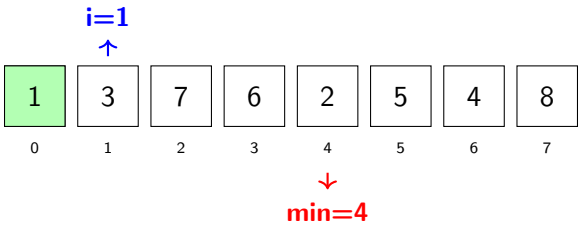
Comparar: a[j=4]=2 $<$ a[min=1]=3? Sí $\rightarrow$ min=4



Observa: i marca posición a rellenar, j busca mínimo, min guarda índice del menor

Selection Sort: Animación paso a paso

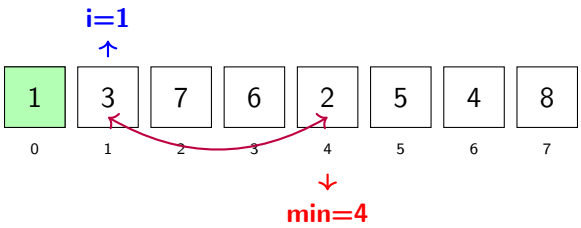
Bucle termina, encontrado min=4 (valor=2)



Observa: i marca posición a rellenar, j busca mínimo, min guarda índice del menor

Selection Sort: Animación paso a paso

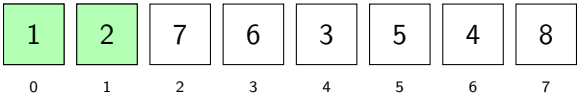
`exch(a, i=1, min=4)` $\rightarrow$ intercambia 3 y 2



Observa: i marca posición a rellenar, j busca mínimo, min guarda índice del menor

Selection Sort: Animación paso a paso

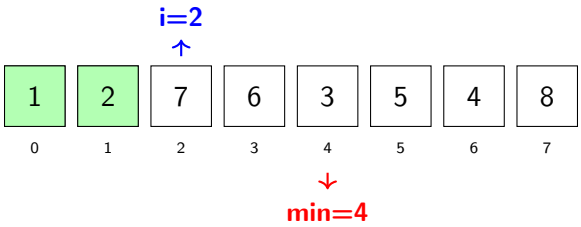
Posiciones 0-1 ordenadas



Observa: i marca posición a rellenar, j busca mínimo, min guarda índice del menor

Selection Sort: Animación paso a paso

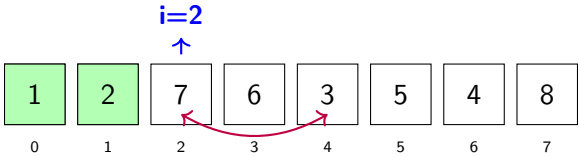
`i=2`: buscar mínimo → min=4 (valor=3)



Observa: i marca posición a rellenar, j busca mínimo, min guarda índice del menor

Selection Sort: Animación paso a paso

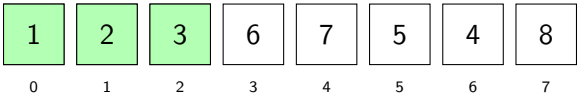
`exch(a, 2, 4)` → intercambia 7 y 3



Observa: i marca posición a rellenar, j busca mínimo, min guarda índice del menor

Selection Sort: Animación paso a paso

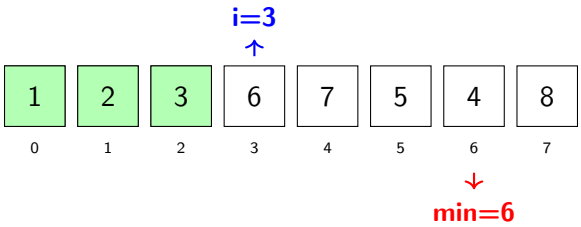
Posiciones 0-2 ordenadas



Observa: i marca posición a rellenar, j busca mínimo, min guarda índice del menor

Selection Sort: Animación paso a paso

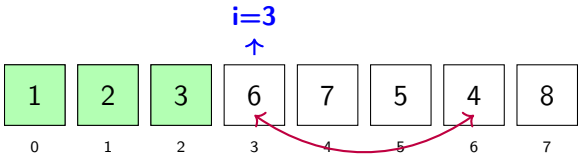
`i=3`: buscar mínimo → min=6 (valor=4)



Observa: i marca posición a rellenar, j busca mínimo, min guarda índice del menor

Selection Sort: Animación paso a paso

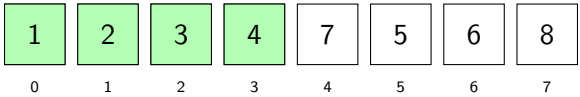
`exch(a, 3, 6)` → intercambia 6 y 4



Observa: i marca posición a rellenar, j busca mínimo, min guarda índice del menor

Selection Sort: Animación paso a paso

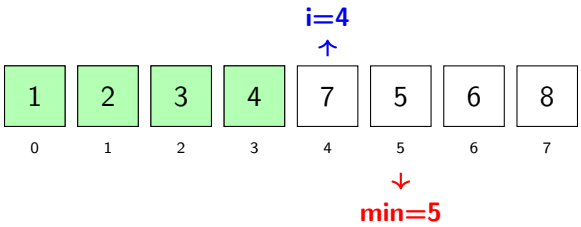
Posiciones 0-3 ordenadas



Observa: i marca posición a rellenar, j busca mínimo, min guarda índice del menor

Selection Sort: Animación paso a paso

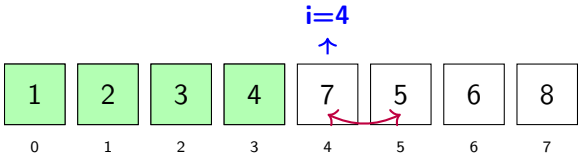
`i=4`: buscar mínimo → min=5 (valor=5)



Observa: i marca posición a rellenar, j busca mínimo, min guarda índice del menor

Selection Sort: Animación paso a paso

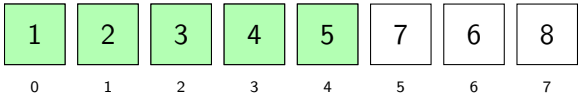
`exch(a, 4, 5)` → intercambia 7 y 5



Observa: i marca posición a rellenar, j busca mínimo, min guarda índice del menor

Selection Sort: Animación paso a paso

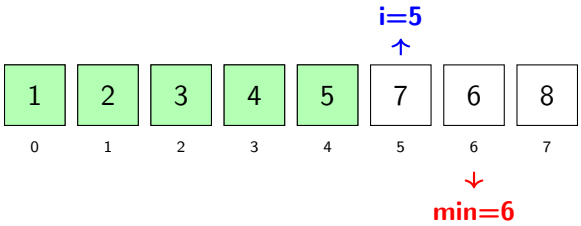
Posiciones 0-4 ordenadas



Observa: i marca posición a rellenar, j busca mínimo, min guarda índice del menor

Selection Sort: Animación paso a paso

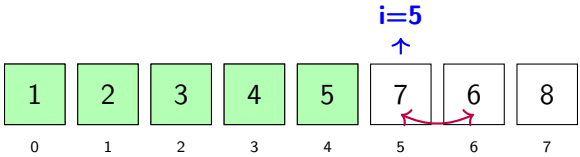
`i=5`: buscar mínimo → min=6 (valor=6)



Observa: i marca posición a rellenar, j busca mínimo, min guarda índice del menor

Selection Sort: Animación paso a paso

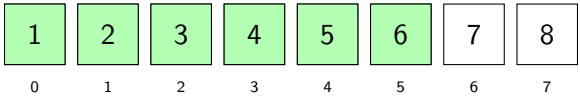
`exch(a, 5, 6)` → intercambia 7 y 6



Observa: i marca posición a rellenar, j busca mínimo, min guarda índice del menor

Selection Sort: Animación paso a paso

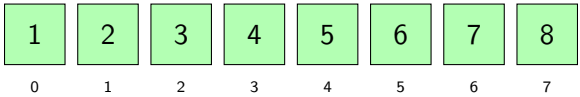
Posiciones 0-5 ordenadas (i=6,7 ya están en su sitio)



Observa: i marca posición a rellenar, j busca mínimo, min guarda índice del menor

Selection Sort: Animación paso a paso

¡Array completamente ordenado!



Observa: i marca posición a rellenar, j busca mínimo, min guarda índice del menor

Selection Sort: análisis matemático

Proposición A:
Selection Sort usa exactamente:

- **Comparaciones:** $(N-1)+(N-2)+\ldots+1+0=\frac{N(N-1)}{2}\sim\frac{N^2}{2}$
- **Intercambios:** N

Demostración:

- En iteración i, comparamos el elemento en i con los $N-1-i$ elementos a su derecha
- Total de comparaciones: $\sum_{i=0}^{N-1}(N-1-i)=\frac{N(N-1)}{2}$
- Hacemos exactamente un intercambio por cada iteración del bucle externo

Características:

- Tiempo de ejecución insensible a la entrada: **siempre cuadrático**
- Movimiento de datos mínimo: número lineal de intercambios

Insertion Sort

Insertion Sort: idea

Analogía: Como ordenar cartas en la mano

- Tomar una carta nueva
- Insertarla en su posición correcta entre las cartas ya ordenadas
- Mover cartas mayores hacia la derecha para hacer espacio

Algoritmo:

- En la iteración i, intercambiar `a[i]` con cada entrada mayor a su izquierda
- Continuar hasta que se encuentre un elemento menor o se llegue al inicio

Insertion Sort: invariantes

Invariantes del algoritmo:

- Las entradas a la **izquierda** del puntero (incluyéndolo):
  - Están en orden ascendente
  - NO necesariamente en su posición final
- Las entradas a la **derecha** del puntero:
  - Aún no han sido vistas



Insertion Sort: implementación

```
public class Insertion
{
    public static void sort(Comparable[] a)
    {
        int N = a.length;
        for (int i = 0; i < N; i++)
            for (int j = i; j > 0; j--)
                if (less(a[j], a[j-1]))
                    exch(a, j, j-1);
                else break;
    }

    private static boolean less(Comparable v, Comparable w)
    { return v.compareTo(w) < 0; }

    private static void exch(Comparable[] a, int i, int j)
    { Comparable temp = a[i]; a[i] = a[j]; a[j] = temp; }
}
```

Nota: El `break` es importante para la eficiencia

Insertion Sort: bucle interno

Para mantener los invariantes:

Paso 1: Mover el puntero a la derecha

- `i++;`

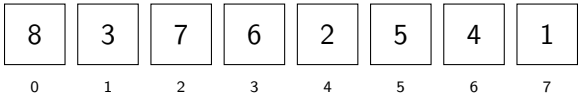
Paso 2: De derecha a izquierda, intercambiar `a[i]` con cada entrada mayor

```
for (int j = i; j > 0; j--)
    if (less(a[j], a[j-1]))
        exch(a, j, j-1);
    else break;
```

Paramos cuando encontramos un elemento menor o llegamos al inicio

Insertion Sort: Animación paso a paso

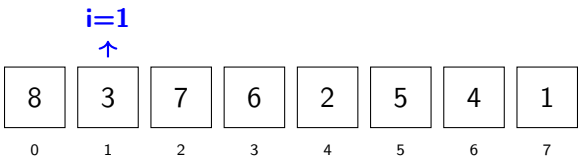
Estado inicial (i=0 ya está ordenado)



Observa: i marca elemento a insertar, j retrocede intercambiando hasta encontrar posición

Insertion Sort: Animación paso a paso

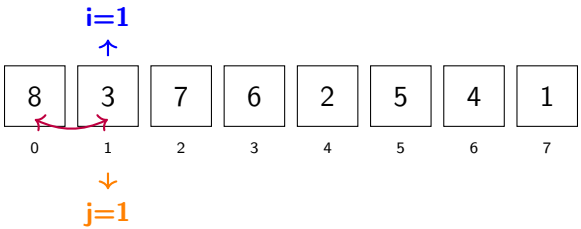
Insertar a[1]=3 en su posición



Observa: i marca elemento a insertar, j retrocede intercambiando hasta encontrar posición

Insertion Sort: Animación paso a paso

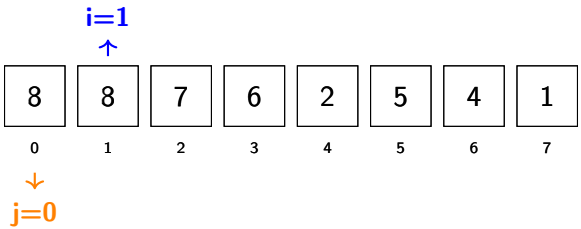
j=1: ¿3 < 8? Sí → intercambio



Observa: i marca elemento a insertar, j retrocede intercambiando hasta encontrar posición

Insertion Sort: Animación paso a paso

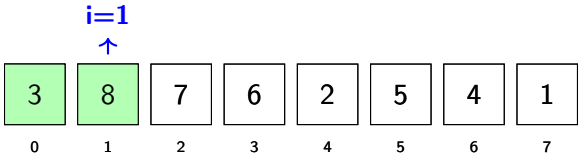
j=0: bucle termina (j>0 falso)



Observa: i marca elemento a insertar, j retrocede intercambiando hasta encontrar posición

Insertion Sort: Animación paso a paso

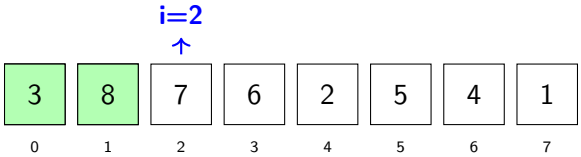
Primeros 2 elementos ordenados



Observa: i marca elemento a insertar, j retrocede intercambiando hasta encontrar posición

Insertion Sort: Animación paso a paso

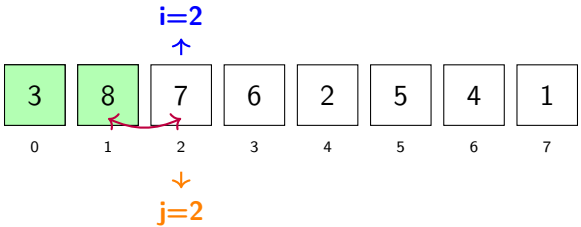
Insertar a[2]=7 en su posición



Observa: i marca elemento a insertar, j retrocede intercambiando hasta encontrar posición

Insertion Sort: Animación paso a paso

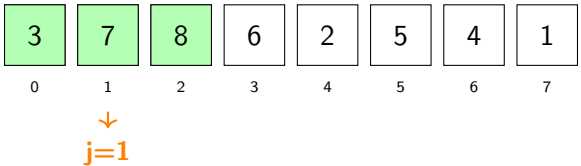
j=2: ¿$7 < 8$? Sí $\rightarrow$ intercambio



Observa: i marca elemento a insertar, j retrocede intercambiando hasta encontrar posición

Insertion Sort: Animación paso a paso

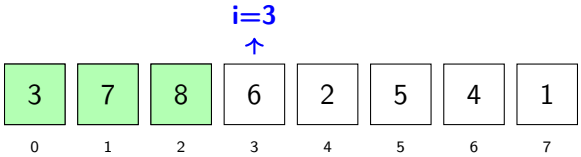
j=1: ¿7 $<$ 3? No $\rightarrow$ para



Observa: i marca elemento a insertar, j retrocede intercambiando hasta encontrar posición

Insertion Sort: Animación paso a paso

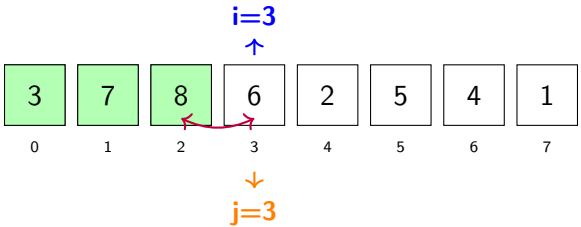
Insertar a[3]=6 en su posición



Observa: i marca elemento a insertar, j retrocede intercambiando hasta encontrar posición

Insertion Sort: Animación paso a paso

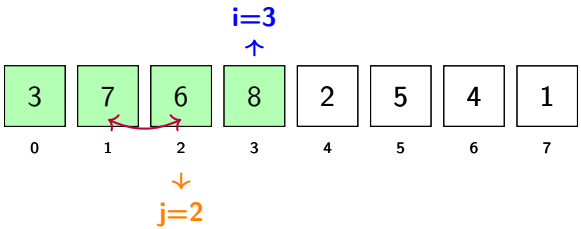
j=3: ¿6 $<$ 8? Sí $\rightarrow$ intercambio



Observa: i marca elemento a insertar, j retrocede intercambiando hasta encontrar posición

Insertion Sort: Animación paso a paso

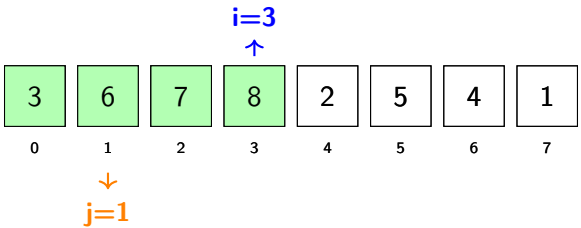
j=2: ¿6 $<$ 7? Sí $\rightarrow$ intercambio



Observa: i marca elemento a insertar, j retrocede intercambiando hasta encontrar posición

Insertion Sort: Animación paso a paso

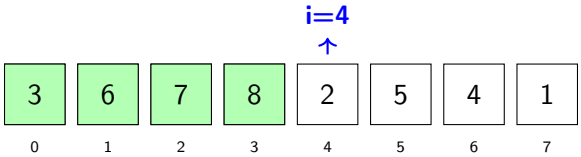
j=1: ¿6 $<$ 3? No $\rightarrow$ para



Observa: i marca elemento a insertar, j retrocede intercambiando hasta encontrar posición

Insertion Sort: Animación paso a paso

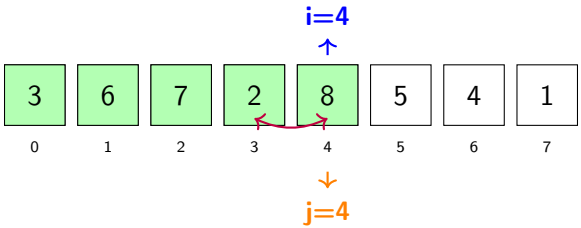
Insertar a[4]=2 (¡recorre mucho!)



Observa: i marca elemento a insertar, j retrocede intercambiando hasta encontrar posición

Insertion Sort: Animación paso a paso

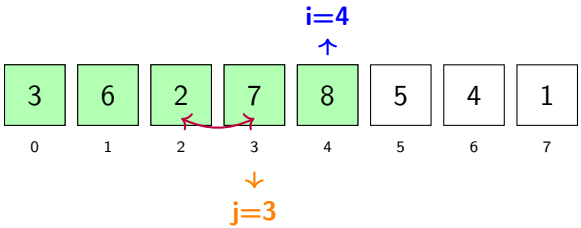
j=4: ¿2 $<$ 8? Sí $\rightarrow$ intercambio



Observa: i marca elemento a insertar, j retrocede intercambiando hasta encontrar posición

Insertion Sort: Animación paso a paso

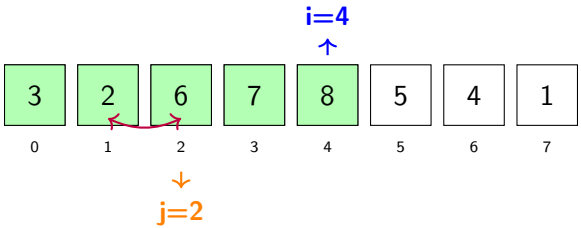
j=3: ¿2 $<$ 7? Sí $\rightarrow$ intercambio



Observa: i marca elemento a insertar, j retrocede intercambiando hasta encontrar posición

Insertion Sort: Animación paso a paso

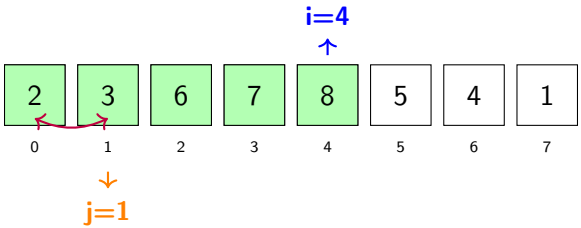
j=2: ¿2 $<$ 6? Sí $\rightarrow$ intercambio



Observa: i marca elemento a insertar, j retrocede intercambiando hasta encontrar posición

Insertion Sort: Animación paso a paso

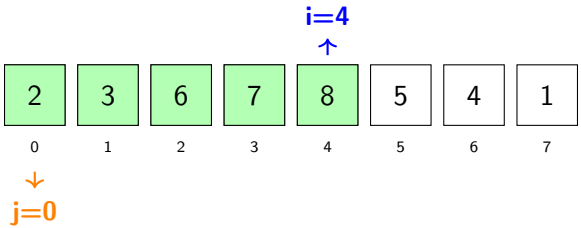
j=1: ¿2 $<$ 3? Sí $\rightarrow$ intercambio



Observa: i marca elemento a insertar, j retrocede intercambiando hasta encontrar posición

Insertion Sort: Animación paso a paso

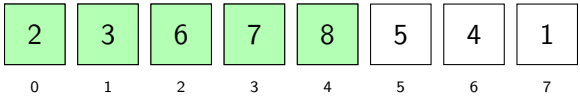
j=0: para (j¿0 falso)



Observa: i marca elemento a insertar, j retrocede intercambiando hasta encontrar posición

Insertion Sort: Animación paso a paso

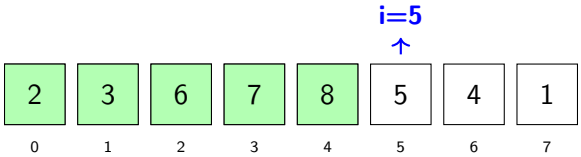
Primeros 5 elementos ordenados



Observa: i marca elemento a insertar, j retrocede intercambiando hasta encontrar posición

Insertion Sort: Animación paso a paso

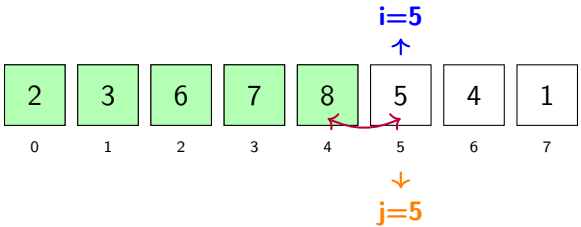
Insertar a[5]=5 en su posición



Observa: i marca elemento a insertar, j retrocede intercambiando hasta encontrar posición

Insertion Sort: Animación paso a paso

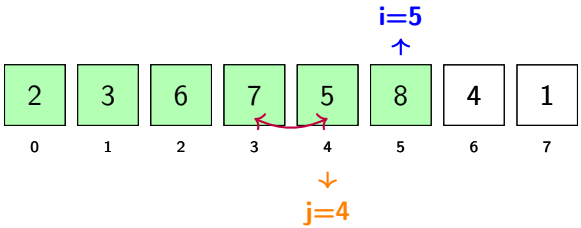
j=5: ¿5 $<$ 8? Sí $\rightarrow$ intercambio



Observa: i marca elemento a insertar, j retrocede intercambiando hasta encontrar posición

Insertion Sort: Animación paso a paso

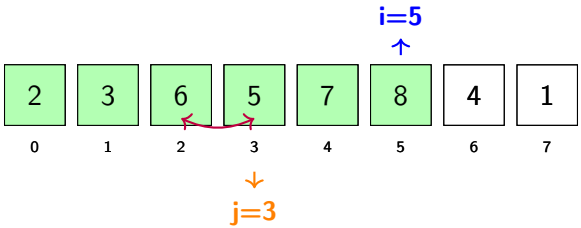
j=4: ¿5 $<$ 7? Sí $\rightarrow$ intercambio



Observa: i marca elemento a insertar, j retrocede intercambiando hasta encontrar posición

Insertion Sort: Animación paso a paso

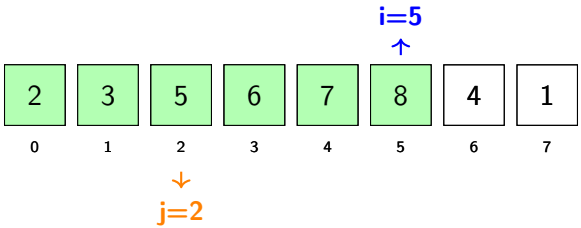
j=3: ¿5 $<$ 6? Sí $\rightarrow$ intercambio



Observa: i marca elemento a insertar, j retrocede intercambiando hasta encontrar posición

Insertion Sort: Animación paso a paso

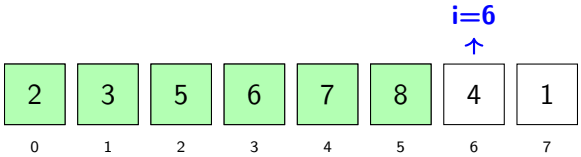
j=2: ¿5 $<$ 3? No $\rightarrow$ para



Observa: i marca elemento a insertar, j retrocede intercambiando hasta encontrar posición

Insertion Sort: Animación paso a paso

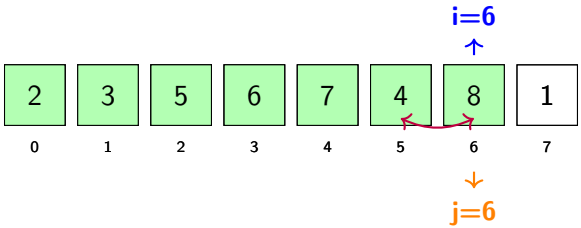
Insertar a[6]=4 en su posición



Observa: i marca elemento a insertar, j retrocede intercambiando hasta encontrar posición

Insertion Sort: Animación paso a paso

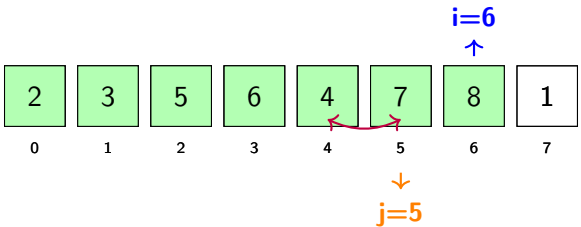
j=6: ¿4 $<$ 8? Sí $\rightarrow$ intercambio



Observa: i marca elemento a insertar, j retrocede intercambiando hasta encontrar posición

Insertion Sort: Animación paso a paso

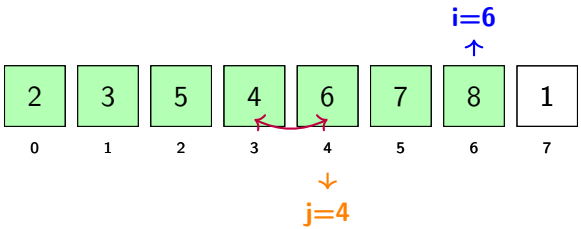
j=5: ¿4 $<$ 7? Sí $\rightarrow$ intercambio



Observa: i marca elemento a insertar, j retrocede intercambiando hasta encontrar posición

Insertion Sort: Animación paso a paso

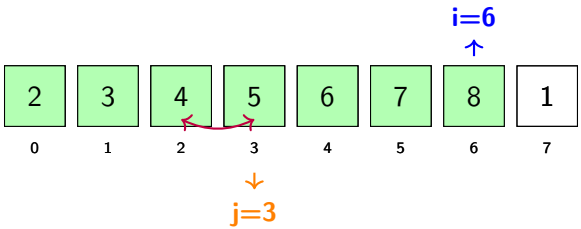
j=4: ¿4 $<$ 6? Sí $\rightarrow$ intercambio



Observa: i marca elemento a insertar, j retrocede intercambiando hasta encontrar posición

Insertion Sort: Animación paso a paso

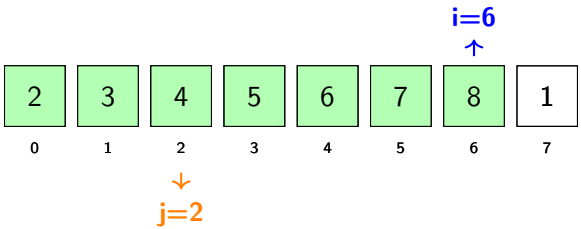
j=3: ¿4 $<$ 5? Sí $\rightarrow$ intercambio



Observa: i marca elemento a insertar, j retrocede intercambiando hasta encontrar posición

Insertion Sort: Animación paso a paso

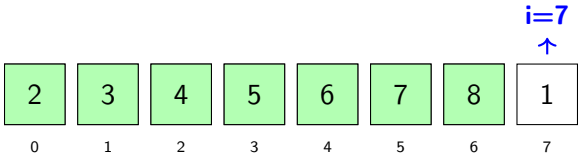
j=2: ¿4 $<$ 3? No $\rightarrow$ para



Observa: i marca elemento a insertar, j retrocede intercambiando hasta encontrar posición

Insertion Sort: Animación paso a paso

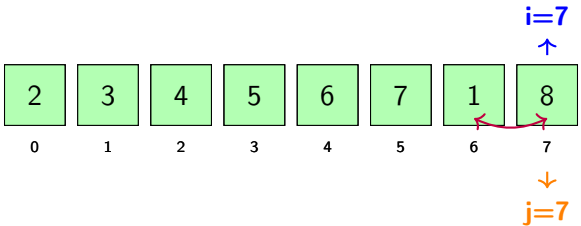
Insertar a[7]=1 (¡recorrerá todo!)



Observa: i marca elemento a insertar, j retrocede intercambiando hasta encontrar posición

Insertion Sort: Animación paso a paso

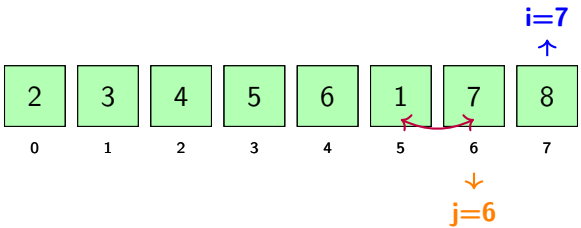
j=7: ¿1 $<$ 8? Sí $\rightarrow$ intercambio



Observa: i marca elemento a insertar, j retrocede intercambiando hasta encontrar posición

Insertion Sort: Animación paso a paso

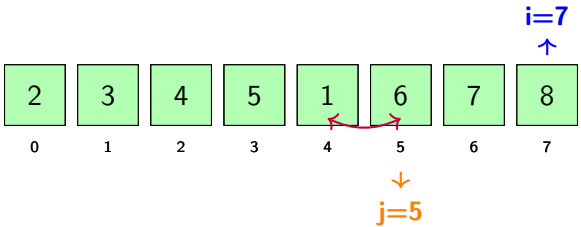
j=6: ¿1 $<$ 7? Sí $\rightarrow$ intercambio



Observa: i marca elemento a insertar, j retrocede intercambiando hasta encontrar posición

Insertion Sort: Animación paso a paso

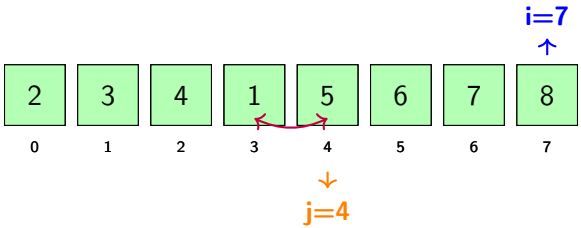
j=5: ¿1 $<$ 6? Sí $\rightarrow$ intercambio



Observa: i marca elemento a insertar, j retrocede intercambiando hasta encontrar posición

Insertion Sort: Animación paso a paso

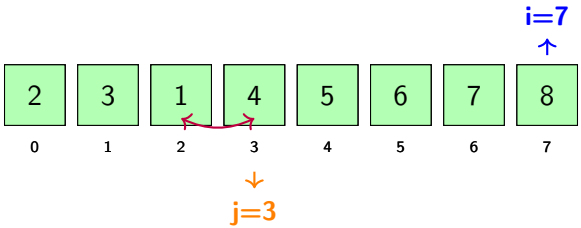
j=4: ¿1 < 5? Sí → intercambio



Observa: i marca elemento a insertar, j retrocede intercambiando hasta encontrar posición

Insertion Sort: Animación paso a paso

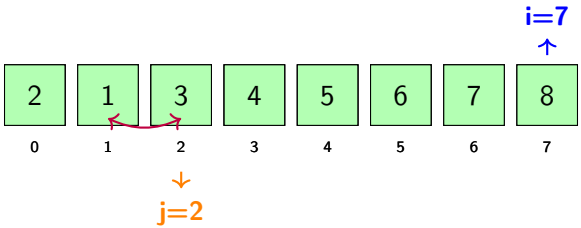
j=3: ¿1 $<$ 4? Sí $\rightarrow$ intercambio



Observa: i marca elemento a insertar, j retrocede intercambiando hasta encontrar posición

Insertion Sort: Animación paso a paso

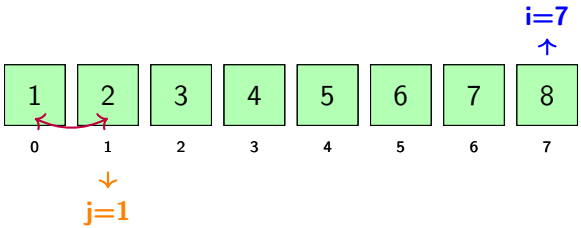
j=2: ¿1 $<$ 3? Sí $\rightarrow$ intercambio



Observa: i marca elemento a insertar, j retrocede intercambiando hasta encontrar posición

Insertion Sort: Animación paso a paso

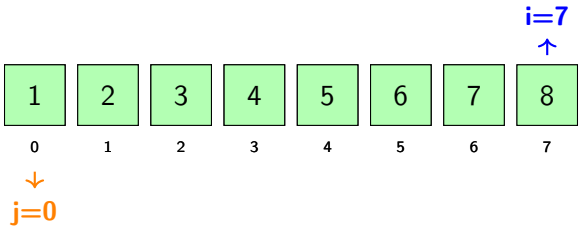
j=1: ¿1 $<$ 2? Sí $\rightarrow$ intercambio



Observa: i marca elemento a insertar, j retrocede intercambiando hasta encontrar posición

Insertion Sort: Animación paso a paso

j=0: para (j>0 falso)



Observa: i marca elemento a insertar, j retrocede intercambiando hasta encontrar posición

Insertion Sort: análisis matemático

Proposición B:
Para ordenar un array aleatorio con claves distintas, Insertion Sort usa:

- **Comparaciones:** $\sim \frac{N^2}{4}$ en promedio
- **Intercambios:** $\sim \frac{N^2}{4}$ en promedio

Demostración (intuitiva):

- En promedio, cada elemento se mueve hacia atrás hasta la **mitad** del camino
- Total (aplicando la fórmula de la suma aritmética $\sum_{i=a}^{K} i = \frac{K(a+K)}{2}$):

$$\sum_{i=1}^{N-1} \frac{i}{2} = \frac{1}{2} \sum_{i=1}^{N-1} i = \frac{1}{2} \cdot \frac{(N-1)N}{2} = \frac{N(N-1)}{4} \approx \frac{N^2}{4}$$

Casos extremos:

- **Mejor caso (array ordenado):** $N-1$ comparaciones, 0 intercambios
- **Peor caso (array invertido):** $\sim \frac{N^2}{2}$ comparaciones e intercambios

Arrays parcialmente ordenados

Definición: Una **inversión** es un par de claves fuera de orden

Ejemplo: `A E E L M O T R X P S`

- Inversiones: T-R, T-P, T-S, R-P, X-P, X-S (6 inversiones)

Definición: Un array está **parcialmente ordenado** si el número de inversiones es $\leq c \cdot N$

Ejemplos de arrays parcialmente ordenados:

- Array completamente ordenado (0 inversiones)
- Subarray pequeño añadido a un array ordenado
- Array con solo unos pocos elementos fuera de lugar (hay subarrays ordenados)

Proposición C: Para arrays **parcialmente ordenados**, Insertion Sort se ejecuta en **tiempo lineal**

Número de intercambios = número de inversiones

Mejoras prácticas para Insertion Sort (1/2)

Mejora 1: Desplazar en lugar de intercambiar

Insertion Sort clásico: en cada paso del bucle interno **intercambia** el elemento con su vecino izquierdo (escribe dos veces por par).

Idea: en vez de intercambiar, **guardar** el elemento a insertar en una variable temporal y **desplazar** los mayores un hueco a la derecha. Al final, colocar el elemento en el hueco libre.

Con intercambios:

- Por cada paso: **2 escrituras** (`exch`)
- j=4: swap(a[4],a[3]), swap(a[3],a[2]), ...

Con desplazamiento:

- `tmp = a[i]`
- Por cada paso: **1 escritura** (desplazar)
- Al final: `a[j+1] = tmp`

$\Rightarrow$ Aprox. **la mitad de escrituras** en el bucle interno. Pero ya no se puede implementar solo con `less()` y `exch()`.

Ojo: la complejidad sigue siendo $\mathcal{O}(N^2)$. Solo se mejora la **constante**: los desplazamientos siguen siendo $\sim \frac{N^2}{4}$, simplemente cada uno cuesta la mitad.

Mejoras prácticas para Insertion Sort (2/2)

Mejora 2: Búsqueda binaria

La parte izquierda `a[0..i-1]` ya está ordenada $\Rightarrow$ se puede usar búsqueda binaria para encontrar la **posición de inserción**.

¿Cómo? No busca un valor exacto (puede que no exista). Busca el índice `pos` tal que:

$$\texttt{a[pos-1]} \leq \texttt{key} < \texttt{a[pos]}$$

Es decir, el primer índice tal que `a[pos] > key`. La búsqueda termina cuando `lo == hi`: ese es el hueco.

¿Qué mejora?

- Comparaciones: de $\sim \frac{N^2}{4}$ a $\sim N \lg N$

¿Qué NO mejora?

- Los **desplazamientos**: siguen siendo $\sim \frac{N^2}{4}$
- Hay que mover todos los elementos entre `pos` e `i`

$\Rightarrow$ El tiempo sigue dominado por los desplazamientos $\Rightarrow$ sigue siendo $\mathcal{O}(N^2)$.

Shellsort

Motivación: problema de Insertion Sort

Problema de Insertion Sort:

- Los elementos se mueven **una posición en cada iteración** (de uno en uno)
- Si un elemento pequeño está al final, se necesitan muchos movimientos
- Ejemplo: array invertido requiere $\mathcal{O}(N^2)$ movimientos

Idea de Shellsort:

- Mover elementos **más de una posición** a la vez
- Primero ordenar elementos lejanos entre sí
- Gradualmente reducir la distancia
- Terminar con Insertion Sort estándar (distancia 1)

Resultado:

- El Insertion Sort que se hace al final es muy eficiente (array casi ordenado)
- Tiempo de ejecución **subcuadrático**: estrictamente menor que $\mathcal{O}(N^2)$

h-sorting

Definición: Un array está **h-ordenado** si al tomar cada *h*-ésimo elemento, obtenemos un array ordenado

Equivalentemente:

- Un array h-ordenado consiste de *h* subarrays intercalados ordenados

Ejemplo con h=4:



¿Cómo h-ordenar un array?

- Usar Insertion Sort con saltos de longitud *h*

Shellsort: algoritmo

Algoritmo de Shellsort [Donald Shell, 1959]:

1. Elegir una secuencia decreciente de valores h: $h_1, h_2, \ldots, h_t$
2. h-ordenar el array para cada h_i en la secuencia
3. El último valor debe ser $h_t = 1$ (Insertion Sort estándar)

¿Por qué funciona?

- Un array que está h-ordenado permanece h-ordenado después de g-ordenarlo
- Cuando llegamos a 1-ordenar (Insertion Sort), el array está parcialmente ordenado
- Insertion Sort es muy eficiente en arrays parcialmente ordenados

Ejemplo de secuencia h: $h = 13, 4, 1$

$$\texttt{13-sort} \rightarrow \texttt{4-sort} \rightarrow \texttt{1-sort}$$

Shellsort: implementación

```
public class Shell
{
    public static void sort(Comparable[] a)
    {
        int N = a.length;

        // Computar incremento: 1, 4, 13, 40, 121, 364, ...
        int h = 1;
        while (h < N/3) h = 3*h + 1;

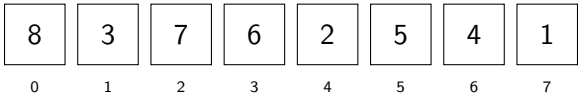
        while (h >= 1)
        {
            // h-ordenar el array
            for (int i = h; i < N; i++)
                for (int j = i; j >= h && less(a[j], a[j-h]); j -= h)
                    exch(a, j, j-h);

            h = h/3;  // reducir incremento
        }
    }
}
```

Shellsort: Animación paso a paso

Array: `[8, 3, 7, 6, 2, 5, 4, 1]` **Secuencia:** `h = 4, 1`

Estado inicial



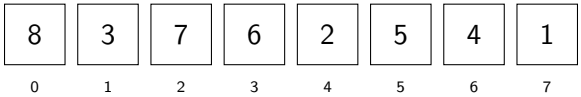
Clave: h-sorting con h grande prepara el array, el 1-sort final requiere pocos movimientos

Shellsort: Animación paso a paso

Array: `[8, 3, 7, 6, 2, 5, 4, 1]` **Secuencia:** `h = 4, 1`

Con h=4 hay 4 subsecuencias intercaladas

h=4

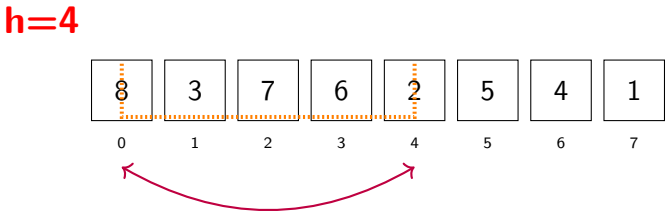


Clave: h-sorting con h grande prepara el array, el 1-sort final requiere pocos movimientos

Shellsort: Animación paso a paso

Array: `[8, 3, 7, 6, 2, 5, 4, 1]` **Secuencia:** `h = 4, 1`

Subsec. 0 (pos. 0,4): [8,2] → 8>2 ¡intercambiar!



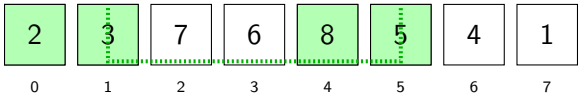
Clave: h-sorting con h grande prepara el array, el 1-sort final requiere pocos movimientos

Shellsort: Animación paso a paso

Array: `[8, 3, 7, 6, 2, 5, 4, 1]` **Secuencia:** `h = 4, 1`

Subsec. 0: [2,8] ordenada. Subsec. 1 (pos. 1,5): [3,5] ya ordenada, sin intercambio

h=4



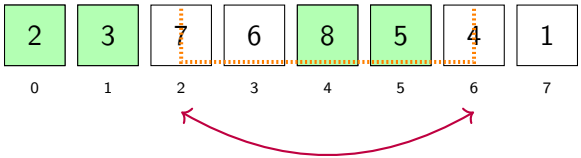
Clave: h-sorting con h grande prepara el array, el 1-sort final requiere pocos movimientos

Shellsort: Animación paso a paso

Array: `[8, 3, 7, 6, 2, 5, 4, 1]` **Secuencia:** `h = 4, 1`

Subsec. 2 (pos. 2,6): [7,4] → 7>4 ¡intercambiar!

h=4



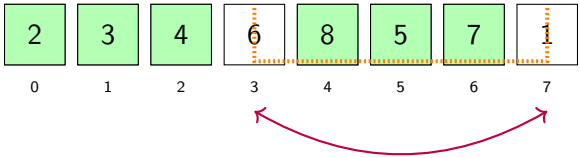
Clave: h-sorting con h grande prepara el array, el 1-sort final requiere pocos movimientos

Shellsort: Animación paso a paso

Array: `[8, 3, 7, 6, 2, 5, 4, 1]` **Secuencia:** `h = 4, 1`

Subsec. 2: [4,7] ordenada. Subsec. 3 (pos. 3,7): [6,1] $\rightarrow$ 6>1 ¡intercambiar!

h=4



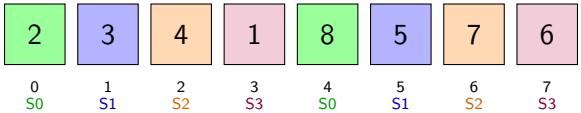
Clave: h-sorting con h grande prepara el array, el 1-sort final requiere pocos movimientos

Shellsort: Animación paso a paso

Array: `[8, 3, 7, 6, 2, 5, 4, 1]` **Secuencia:** `h = 4, 1`

Array 4-ordenado: cada color es una subsecuencia de paso 4 (cada una está ordenada)

h=4



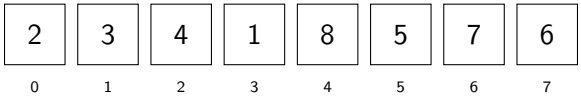
Clave: h-sorting con h grande prepara el array, el 1-sort final requiere pocos movimientos

Shellsort: Animación paso a paso

Array: `[8, 3, 7, 6, 2, 5, 4, 1]` **Secuencia:** `h = 4, 1`

Ahora **h=1**: Insertion Sort normal sobre array **casi ordenado**

h=1

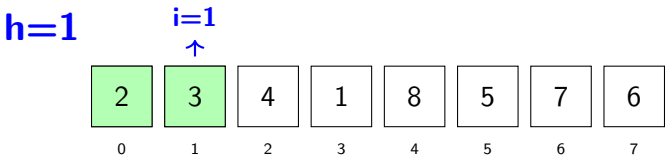


Clave: h-sorting con h grande prepara el array, el 1-sort final requiere pocos movimientos

Shellsort: Animación paso a paso

Array: `[8, 3, 7, 6, 2, 5, 4, 1]` **Secuencia:** `h = 4, 1`

i=1: a[1]=3 $\geq$ a[0]=2 $\Rightarrow$ sin intercambio

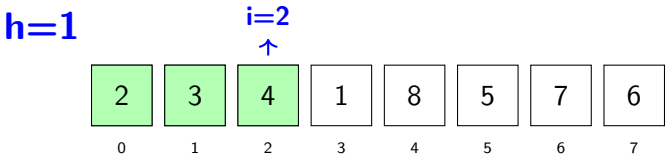


Clave: h-sorting con h grande prepara el array, el 1-sort final requiere pocos movimientos

Shellsort: Animación paso a paso

Array: `[8, 3, 7, 6, 2, 5, 4, 1]` **Secuencia:** `h = 4, 1`

i=2: a[2]=4 $\geq$ a[1]=3 $\Rightarrow$ sin intercambio

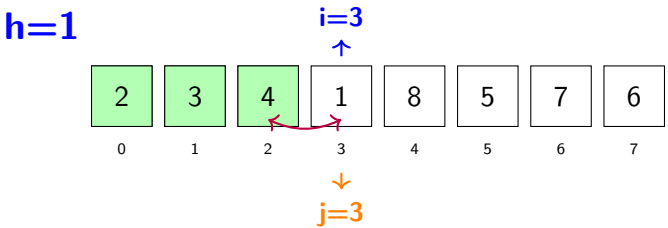


Clave: h-sorting con h grande prepara el array, el 1-sort final requiere pocos movimientos

Shellsort: Animación paso a paso

Array: `[8, 3, 7, 6, 2, 5, 4, 1]` **Secuencia:** `h = 4, 1`

i=3: a[3]=1. j=3: $1 < 4 \Rightarrow$ intercambio

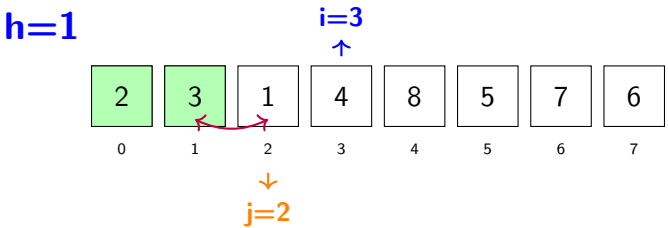


Clave: h-sorting con h grande prepara el array, el 1-sort final requiere pocos movimientos

Shellsort: Animación paso a paso

Array: `[8, 3, 7, 6, 2, 5, 4, 1]` **Secuencia:** `h = 4, 1`

i=3: a[3]=1. j=2: $1 < 3 \Rightarrow$ intercambio

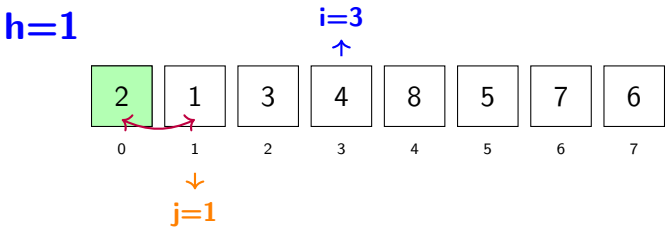


Clave: h-sorting con h grande prepara el array, el 1-sort final requiere pocos movimientos

Shellsort: Animación paso a paso

Array: `[8, 3, 7, 6, 2, 5, 4, 1]` **Secuencia:** `h = 4, 1`

i=3: a[3]=1. j=1: $1 < 2 \Rightarrow$ intercambio



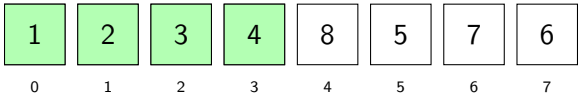
Clave: h-sorting con h grande prepara el array, el 1-sort final requiere pocos movimientos

Shellsort: Animación paso a paso

Array: `[8, 3, 7, 6, 2, 5, 4, 1]` **Secuencia:** `h = 4, 1`

i=3: 1 colocado. j=0: inicio del array, para. [1,2,3,4] ordenado

h=1

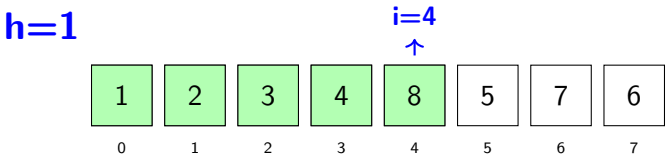


Clave: h-sorting con h grande prepara el array, el 1-sort final requiere pocos movimientos

Shellsort: Animación paso a paso

Array: `[8, 3, 7, 6, 2, 5, 4, 1]` **Secuencia:** `h = 4, 1`

i=4: a[4]=8 $\geq$ a[3]=4 $\Rightarrow$ sin intercambio

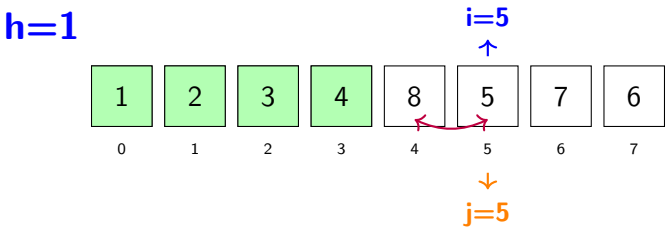


Clave: h-sorting con h grande prepara el array, el 1-sort final requiere pocos movimientos

Shellsort: Animación paso a paso

Array: `[8, 3, 7, 6, 2, 5, 4, 1]` **Secuencia:** `h = 4, 1`

i=5: a[5]=5. j=5: $5 < 8 \Rightarrow$ intercambio



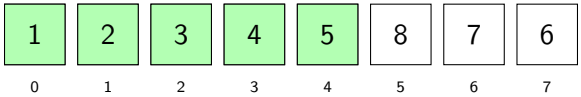
Clave: h-sorting con h grande prepara el array, el 1-sort final requiere pocos movimientos

Shellsort: Animación paso a paso

Array: `[8, 3, 7, 6, 2, 5, 4, 1]` **Secuencia:** `h = 4, 1`

i=5: 5 colocado. j=4: 5 $>$ 4, para. [1,2,3,4,5,8] ordenado

h=1

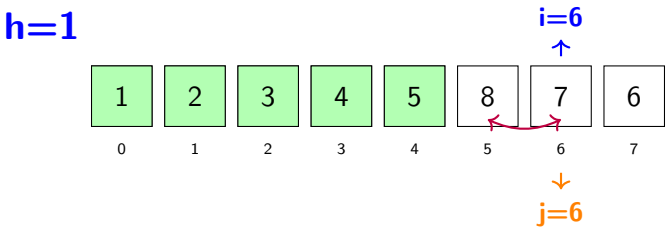


Clave: h-sorting con h grande prepara el array, el 1-sort final requiere pocos movimientos

Shellsort: Animación paso a paso

Array: `[8, 3, 7, 6, 2, 5, 4, 1]` **Secuencia:** `h = 4, 1`

i=6: a[6]=7. j=6: $7 < 8 \Rightarrow$ intercambio



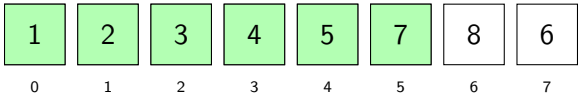
Clave: h-sorting con h grande prepara el array, el 1-sort final requiere pocos movimientos

Shellsort: Animación paso a paso

Array: `[8, 3, 7, 6, 2, 5, 4, 1]` **Secuencia:** `h = 4, 1`

i=6: 7 colocado. j=5: 7 $>$ 5, para. [1,2,3,4,5,7,8] ordenado

h=1

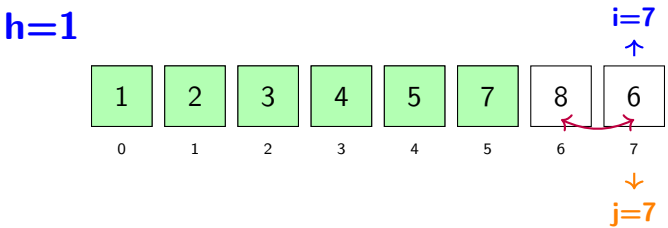


Clave: h-sorting con h grande prepara el array, el 1-sort final requiere pocos movimientos

Shellsort: Animación paso a paso

Array: `[8, 3, 7, 6, 2, 5, 4, 1]` **Secuencia:** `h = 4, 1`

i=7: a[7]=6. j=7: $6 < 8 \Rightarrow$ intercambio

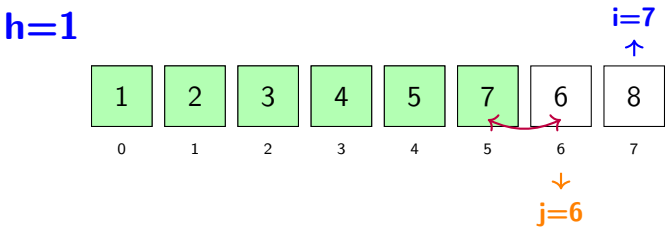


Clave: h-sorting con h grande prepara el array, el 1-sort final requiere pocos movimientos

Shellsort: Animación paso a paso

Array: `[8, 3, 7, 6, 2, 5, 4, 1]` **Secuencia:** `h = 4, 1`

i=7: a[7]=6. j=6: $6 < 7 \Rightarrow$ intercambio



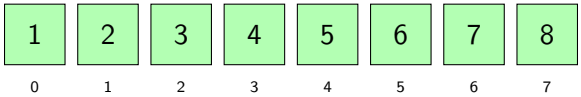
Clave: h-sorting con h grande prepara el array, el 1-sort final requiere pocos movimientos

Shellsort: Animación paso a paso

Array: `[8, 3, 7, 6, 2, 5, 4, 1]` **Secuencia:** `h = 4, 1`

i=7: 6 colocado. j=5: 6 > 5, para. **¡Array ordenado!**

h=1



Clave: h-sorting con h grande prepara el array, el 1-sort final requiere pocos movimientos

Secuencia de incrementos

¿Qué secuencia de h usar?

Secuencia 3x+1 (usada en la implementación anterior):

- $h = 1, 4, 13, 40, 121, 364, 1093, \ldots$
- Fórmula: empezar con $h = 1$, luego $h = 3h + 1$
- Fácil de calcular
- Buen rendimiento en la práctica

Otras secuencias propuestas:

- Potencias de 2: $1, 2, 4, 8, 16, \ldots$ (Mala elección)
- Potencias de 2 menos 1: $1, 3, 7, 15, 31, \ldots$ (Mejor)
- Secuencia de Sedgewick: $1, 5, 19, 41, 109, \ldots$ (Muy buena)

Pregunta abierta:

- ¿Cuál es la mejor secuencia de incrementos?
- *¡Aún no se conoce la respuesta!*

Shellsort: análisis

Complejidad:

- Con secuencia 3x+1: $\mathcal{O}(N^{3/2})$ en el peor caso (según estudios experimentales)
- En la práctica: mucho mejor, cercano a $\mathcal{O}(N \log N)$
- Depende fuertemente de la secuencia de incrementos elegida

Ventajas:

- Algoritmo simple, solo unas pocas líneas de código
- Rendimiento no trivial, mejor que $\mathcal{O}(N^2)$
- No requiere espacio adicional (in-place)
- Útil en la práctica para arrays de tamaño moderado

Desventajas:

- Análisis matemático complejo
- Rendimiento depende de la secuencia de incrementos
- No es óptimo asintóticamente

Conclusiones

Comparación de algoritmos elementales

Algoritmo	**Peor caso**	**Promedio**	**Mejor caso**
Selection Sort	$\mathcal{O}(N^2)$	$\mathcal{O}(N^2)$	$\mathcal{O}(N^2)$
Insertion Sort	$\mathcal{O}(N^2)$	$\mathcal{O}(N^2)$	$\mathcal{O}(N)$
Shellsort	$\mathcal{O}(N^{3/2})$	depende	depende

¿Cuándo usar cada uno?

- **Selection Sort:** Cuando los intercambios son costosos
- **Insertion Sort:** Arrays pequeños o casi ordenados
- **Shellsort:** Arrays medianos, cuando queremos algo simple pero razonable

¿Podemos hacerlo mejor?

- Sí! Algoritmos $\mathcal{O}(N \log N)$: Mergesort, Quicksort, Heapsort
- *Los veremos en las próximas sesiones*

Resumen: Algoritmos elementales

Hemos estudiado:

1. **Interfaz Comparable:** Mecanismo para ordenar cualquier tipo
2. **Selection Sort:** Simple, $\mathcal{O}(N^2)$, mínimo movimiento de datos
3. **Insertion Sort:** $\mathcal{O}(N^2)$, pero eficiente para arrays casi ordenados
4. **Shellsort:** Mejora significativa, $\mathcal{O}(N^{3/2})$ o mejor

Lecciones importantes:

- Los detalles de implementación importan
- Analizar casos mejor, peor y promedio
- Arrays parcialmente ordenados son comunes en la práctica
- Ideas simples pueden llevar a mejoras sustanciales (Shellsort)

¿Qué sigue?

Limitaciones de los algoritmos elementales:

- Selection Sort e Insertion Sort son $\mathcal{O}(N^2)$
- Para arrays grandes, esto es inaceptable
- ¿Podemos hacer mejor?

Próximas sesiones: Algoritmos avanzados de ordenación

- **Mergesort:** $\mathcal{O}(N \log N)$ garantizado
- **Quicksort:** $\mathcal{O}(N \log N)$ en promedio, muy rápido en práctica

Preguntas para discusión

1. ¿Por qué Insertion Sort es mejor que Selection Sort para arrays casi ordenados?
2. ¿Cuál es la intuición detrás de por qué Shellsort funciona?
3. ¿En qué situaciones usarías Selection Sort sobre otros algoritmos?
4. ¿Por qué es importante el número de inversiones en un array?

¿Por qué Insertion Sort es mejor que Selection Sort para arrays casi ordenados?

Insertion Sort:

- El bucle interno se detiene (`break`) cuando encuentra un elemento menor
- En arrays casi ordenados, cada elemento ya está cerca de su posición
- Pocas comparaciones e intercambios por elemento
- Mejor caso: $\mathcal{O}(N)$ cuando el array está ordenado

Selection Sort:

- Siempre examina **todos** los elementos restantes para encontrar el mínimo
- No importa si el array está casi ordenado o completamente desordenado
- Siempre hace $\sim N^2/2$ comparaciones
- No aprovecha el orden parcial existente

Conclusión: Insertion Sort **se adapta** al orden del input, Selection Sort no.

¿Cuál es la intuición detrás de por qué Shellsort funciona?

Problema de Insertion Sort:

- Los elementos se mueven **una posición a la vez**
- Un elemento muy pequeño al final del array necesita $\mathcal{O}(N)$ movimientos

Idea clave de Shellsort:

1. **h-sorting con h grande:** Mueve elementos lejos rápidamente
   - Elementos pequeños al final pueden "saltar" muchas posiciones
   - Elementos grandes al inicio también se mueven lejos
2. **Propiedad 'mágica':** Un array h-ordenado permanece h-ordenado después de g-ordenarlo
   - Cada paso mantiene el progreso de los pasos anteriores
3. **1-sort final:** Como el array ya está casi ordenado, Insertion Sort es muy eficiente

Metáfora: Es como limpiar una habitación: primero mueves los objetos grandes, luego los medianos, y finalmente ajustas los pequeños detalles.

¿En qué situaciones usarías Selection Sort sobre otros algoritmos?

1. **Los intercambios son muy costosos**
   - Selection Sort hace **exactamente N intercambios** (uno por iteración)
   - Insertion Sort puede hacer hasta $\mathcal{O}(N^2)$ intercambios
   - Ejemplo: Mover registros grandes en bases de datos
2. **Necesitas encontrar los k elementos más pequeños**
   - Puedes detener Selection Sort después de k iteraciones
   - Tendrás los k elementos más pequeños en sus posiciones finales
3. **La simplicidad es más importante que el rendimiento**
   - Código muy simple, fácil de entender y verificar
   - Útil para arrays muy pequeños donde N^2 no importa
4. **Quieres comportamiento predecible**
   - Siempre hace el mismo número de comparaciones, sin importar el input
   - No hay mejor ni peor caso (siempre $\mathcal{O}(N^2)$)

¿Por qué es importante el número de inversiones en un array?

Definición: Una **inversión** es un par (i, j) donde $i < j$ pero $a[i] > a[j]$

1. Mide cómo de desordenado está un array:

- Array ordenado: 0 inversiones | Array invertido: $N(N-1)/2$ inversiones
- Array aleatorio: $\sim N^2/4$ inversiones en promedio

2. Relación directa con Insertion Sort:

- Número de intercambios de Insertion Sort = número de inversiones
- Por eso es eficiente en arrays parcialmente ordenados

3. Arrays parcialmente ordenados son comunes en la práctica:

- Datos ordenados con pequeños cambios, unión de secuencias ordenadas, etc.

4. Guía la elección del algoritmo:

- Pocas inversiones ($\leq cN$) $\rightarrow$ Insertion Sort es $\mathcal{O}(N)$
- Muchas inversiones $\rightarrow$ usar algoritmos más avanzados ($\mathcal{O}(N \log N)$)